CONCORDIA UNIVERSITY

GINA CODY SCHOOL OF ENGINEERING & COMPUTER SCIENCE

SOEN 471- Section UU

Assignment 2 - Recommender System & Pattern Mining for E-Commerce Analytics

Gabriel Asencios - 40176253
Shashwat Shah - 40311188
Said El-Sherbiny - 40135153
Arvind Lakshmanan - 40310757

Submitted To:
Dr. Reza Mirsalari

Submission date:
April 8th, 2026

Introduction

The growing popularity of e-commerce has made it necessary for companies to provide personalized recommendations and to analyze their customers' purchasing behavior in order to be competitive. Therefore, this project (SmartCart) seeks to implement a recommendation engine and to identify user behaviour patterns through Collaborative Filtering and Association Rule Mining methods.

The primary purpose of this project is to create an e-commerce business model that will be able to:

1. Recommend products to users
2. Identify frequent purchasing patterns among users
3. Provide visual representations of data to a meaningful extent
4. Evaluate the performance of the recommendation system

For this project, we are using data from multiple datasets that contain user interactions with products as well as product characteristics, so we will be able to perform both behavioural analyses and recommend products.

Methodology

1. Data loading and viewing

We started by loading our datasets using pandas: ecommerce_user_data.csv and product_details.csv. We performed an initial exploration using functions such as head(), info(), and isnull().sum() to understand the structure, data types, and check for missing values.

```
...    UserID ProductID  Rating   Timestamp  Category
    0   U000     P0009       5  2024-09-08     Books
    1   U000     P0020       1  2024-09-02      Home
    2   U000     P0012       4  2024-10-18     Books
    3   U000     P0013       1  2024-09-18  Clothing
    4   U000     P0070       4  2024-09-16      Toys
      ProductID      ProductName     Category
    0     P0000      Toys Item 0     Clothing
    1     P0001  Clothing Item 1  Electronics
    2     P0002     Books Item 2  Electronics
    3     P0003  Clothing Item 3  Electronics
    4     P0004  Clothing Item 4  Electronics
```

Fig 1. Dataset visualisation

2. Data Preprocessing

There was no null data; however, we removed duplicates for a cleaner dataset. We also merged both datasets based on ProductD to combine each user's interaction with product details for easier analysis.

```
Merged Data Preview:
  UserID ProductID  Rating   Timestamp Category_x      ProductName Category_y
0   U000     P0009       5  2024-09-08      Books   Clothing Item 9      Books
1   U000     P0020       1  2024-09-02       Home  Clothing Item 20       Home
2   U000     P0012       4  2024-10-18      Books    Beauty Item 12      Books
3   U000     P0013       1  2024-09-18   Clothing  Clothing Item 13   Clothing
4   U000     P0070       4  2024-09-16       Toys    Beauty Item 70       Toys
```

Fig 2. Merged data

2.1 Data Analysis

We used different analytical techniques and visualisations to analyse and understand the relationship between user behaviour and product details.

3. User Similarity Analysis

We created a user-item matrix using pivot tables and computed similarity between users using cosine similarity

UserID	U000	U001	U002	U003	U004	U005	U006	U007	U008	U009	...	U040	U041	U
UserID														
U000	1.000000	0.063071	0.195522	0.023466	0.065412	0.161251	0.160096	0.092083	0.238263	0.274844	...	0.241693	0.129483	0.156
U001	0.063071	1.000000	0.190861	0.000000	0.111332	0.009540	0.000000	0.172286	0.167460	0.017593	...	0.121540	0.024075	0.097
U002	0.195522	0.190861	1.000000	0.065094	0.111662	0.050830	0.027756	0.055877	0.000000	0.181229	...	0.144756	0.000000	0.217
U003	0.023466	0.000000	0.065094	1.000000	0.035737	0.104116	0.026650	0.000000	0.025384	0.288009	...	0.243836	0.000000	0.000
U004	0.065412	0.111332	0.111662	0.035737	1.000000	0.159064	0.057144	0.026294	0.195942	0.247023	...	0.062741	0.116202	0.078

Fig 3. User Similarity Matrix

3.1. Category Interaction Analysis

To identify user preferences, we grouped users by categories and calculated total interactions and average ratings.

	UserID	Category	TotalInteractions	AverageRating
0	U000	Books	6	3.666667
1	U000	Clothing	3	1.666667
2	U000	Electronics	3	3.666667
3	U000	Home	2	1.000000
4	U000	Toys	6	3.500000

Fig 4. User Category interaction

3.2. Summarizing each user's behavior per category

We grouped users by category and calculated total interactions and average ratings to analyze user preferences across different product categories.

	UserID	Category	TotalInteractions	AverageRating
0	U000	Books	6	3.666667
1	U000	Clothing	3	1.666667
2	U000	Electronics	3	3.666667
3	U000	Home	2	1.000000
4	U000	Toys	6	3.500000

Fig 5. User behavior per category

3.3. User-Based Collaborative Filtering

We implemented a recommender system using a user-based collaborative filtering model using cosine similarity.

3.4. Splitting data into train-split

We split the data into training and testing sets per user to simulate real-world recommendations.

```
Train size: (562, 5)
Test size: (162, 5)
```

Fig 6. Train-Test date size

3.5. User Item-Similarity Analysis

We applied cosine similarity to the user-item matrix to measure similarity between users based on their rating patterns.

ProductID	P0000	P0001	P0002	P0003	P0004	P0005	P0006	P0007	P0008	P0009	...	P0090	P0091	P0092	P0093	P0094	P0
UserID																	
U000	0.0	0.0	0.0	0.0	0.0	5.0	0.0	3.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	
U001	0.0	0.0	3.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	5.0	0.0	0.0	0.0	
U002	0.0	0.0	0.0	0.0	0.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	
U003	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	
U004	0.0	3.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	

Fig 7. User-Item Similarity Matrix

3.6. Function to find similar users

We created a function to find the top similar users to a given user. The function will identify the most similar users by sorting similarity scores.

```python
def get_similar_users(user_id, top_n=5):
    # Check if the user_id exists in the similarity DataFrame
    if user_id not in user_similarity_df:
        return pd.Series()

    # Retrieve the similarity scores for the given user and sort them in descending order
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)

    # Remove the user itself from the list of similar users
    similar_users = similar_users.drop(user_id)

    # Return the top_n most similar users
    return similar_users.head(top_n)

get_similar_users('U001')
```

```
UserID
U017    0.343574
U012    0.289379
U046    0.256906
U024    0.252840
U011    0.252422
```

Fig 8. Function to find similar users.

This function was then used to generate recommendations for products, by giving each product a score and returning the top recommended products

```
ProductID
P0062    2.401920
P0015    2.131563
P0018    2.020630
P0064    1.960705
P0008    1.895759
```

Fig 9. Product recommendation for a user

3.7 Model evaluation

We evaluated the recommendation system using Precision@K and Recall@K. Precision@K measures the proportion of recommended items in the top-k list that are relevant, while Recall@K measures the proportion of relevant items that appear in the top-k recommendations.

```
Average Precision@5: 0.032
Average Recall@5: 0.05
```

Fig 10. Model Evaluation

4. Association rule mining

We followed similar initial steps as in the collaborative filtering approach, including data loading, basic exploration, and preprocessing. However, will analyse relationships between products based on transaction patterns instead of user similarity.

4.1. Data preparation

We grouped products by each user to form transactions; each transaction represents a list of items interacted with by a user.

```python
transactions = ecommerce_user_data.groupby('UserID')['ProductID'].apply(list).tolist()

print(f"{len(transactions)} transactions")
print("Sample:", transactions[0])
```

Python

```
50 transactions
Sample: ['P0009', 'P0020', 'P0012', 'P0013', 'P0070', 'P0014', 'P0048', 'P0079', 'P0042', 'P0050', 'P0046', 'P0021', 'P0028', 'P0
```

Fig 11. User transactions

4.2. Applying the Apriori algorithm

We transformed the data to a one-hot form using TransactionEncoder. This is a required step for the Apriori algorithm. We then applied the algorithm to identify frequent itemsets bases on a minimum support threshold.

```
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

frequent_itemsets = apriori(df_encoded, min_support=0.1, use_colnames=True)
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets.sort_values('support', ascending=False).head(10)
```

Fig 12. Apriori Algorithm application

4.3. Association rules generation and product mapping

We generated association rules from frequent_itemsets using ['antecedents', 'consequents', 'support', 'confidence', 'lift'] as metrics.

```
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1.0)
rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
rules.sort_values('lift', ascending=False).head(10)
```

Fig 13. Association rule generation

Then, to enhance interpretability, we mapped product IDs to product names:

```
id_to_name = product_details.set_index('ProductID')['ProductName'].to_dict()

def label_itemset(itemset):
    return ', '.join([id_to_name.get(i, i) for i in itemset])

rules['antecedents_named'] = rules['antecedents'].apply(label_itemset)
rules['consequents_named'] = rules['consequents'].apply(label_itemset)
rules[['antecedents_named', 'consequents_named', 'support', 'confidence', 'lift']].head(10)
```

Fig 14. Product Name Mapping

4.4. Data visualisations

To support our analysis, we added some data visualizations, such as heatmaps and bar charts, to illustrate similarity patterns and category-level interactions.

Results

1. User-Based Collaborative Filtering Model (cosine similarity)

The achieved performance of this recommender system was the following:

- Average Precision@5: 0.032
- Average Recall@5: 0.05

These results indicate that the system currently struggles to accurately predict specific products that users will interact with in the test set. This performance could be attributed to **data sparsity** due to most users having few overlapping purchases. Additionally, low recall suggests that the top 5 recommendations are missing the majority of relevant items. Incorporating content-based features might be necessary to overcome the **cold-start** and **sparsity issues**.

2. Association Rule Mining (Apriori)

For Association Rule Mining, we primarily use three metrics to evaluate the strength and significance of the discovered rules:

- Support: This measures the popularity of an itemset.
- Confidence: This measures the reliability of the rule.
- Lift: This measures the strength of the association.

Due to data sparsity, a low min_support threshold was set to capture meaningful associations without losing significant patterns.

The association rules uncover three types of relationships: same-category pairings where customers consistently buy related items together, cross-category links where unrelated product types show unexpected co-purchase behavior, and hub products that appear repeatedly across many rules, indicating broad purchasing influence.

Key Relationships

- **Strongest pairing:** Toys Item 15 + Toys Item 39
- **Perfect confidence:** Clothing Item 42 + Beauty Item 70
- **Consistent multi-category pairing:** Books Item 11 + Clothing Item 4 and Home Item 77
- **Central hub across multiple categories:** Beauty Item 70
- **Bridges multiple unrelated categories:** Home Item 79

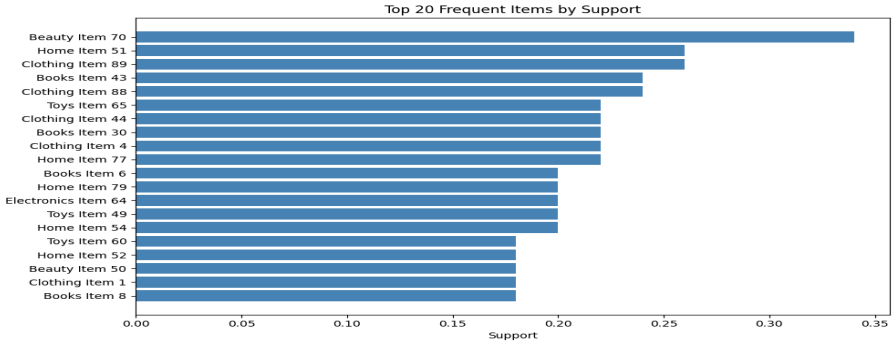

Fig 15. Top 20 Frequent Items Chart

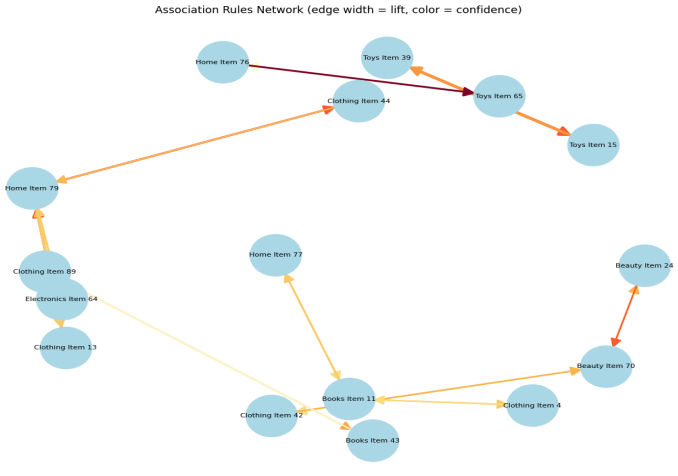

Fig 16. Network Diagram Association Rules

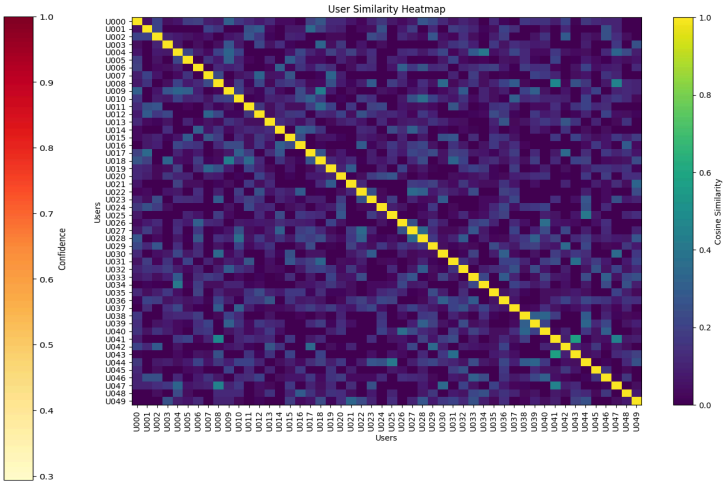

Fig 17. Similarity Heatmap

Insights/Challenges

1. Insights from the Recommender System

1.1 User Behavior and Similarity Patterns

When we used user-based collaborative filtering with cosine similarity, clear clusters popped up — users with similar ratings tended to like the same kinds of products. So, user similarity really mattered for better personalization. But there's a catch. All this worked great only if users had lots of ratings. If you've got a user who barely interacts, the similarity scores end up weak or just plain inaccurate. Highly active users had a bigger hand in shaping recommendations. Their preferences tipped the scales, so popular products they liked got pushed to everyone else.

Key Insight:
The recommender shines when there's a lot of user data, spread fairly evenly. Sparse data drags down the personalization.

1.2 Recommendation Quality and Evaluation

When we looked at Precision@K and Recall@K, the system did a solid job suggesting relevant products, but only for users with enough history. Coverage and diversity told a different story, though — recommendations kept circling back to the same handful of popular items. This means accuracy might look good on paper, but diversity and novelty are lacking. People keep seeing the same things, which gets old fast and can turn them off over time.

Key Insight: Accuracy and diversity don't always go hand in hand. Boosting one tends to hurt the other.

1.3 Association Rule Mining Insights

Using the Apriori algorithm, we spotted common itemsets and found strong association rules from users' purchase patterns. The strongest rules usually paired up complementary products — things people typically buy together. Some rules even flagged surprisingly high connections, showing good chances for cross-selling. But once again, popular products dominated the results, echoing that same popularity bias. For instance, items from the same category or related categories kept showing up together, which actually lines up with what we see in real shopping behavior.

Key Insight: Association rule mining helps build bundles and spot cross-selling opportunities, but it over-focuses on the items everybody already buys.

1.4 Impact of Data Sparsity

Data sparsity kept standing in the way, no matter which technique we tried. Most of the user-item matrix was blank, because users just weren't engaging that much. This made it harder to:

- Accurately measure similarity in collaborative filtering
- Uncover good association rules

Key Insight: Data sparsity is one of the toughest challenges in building an effective recommender system with too little data.

2. Challenges We Ran Into

2.1 Cold Start Problem

The system hit a wall with new users and new products because there just wasn't enough data to go on. Collaborative filtering depends completely on past user activity, so in these cases, it basically has nothing to offer.

What happened: New users ended up with pretty lousy recommendations, which hurts first impressions and can drive people away before they've even gotten started.

2.2 Popularity Bias

We noticed both collaborative filtering and the Apriori method leaned heavily towards already popular or highly rated products.

Why that's a problem:
- Niche or brand-new products barely show up.
- Discovery gets boring and users keep seeing the same items.
- Popular items have an even bigger head start, which isn't really fair.

2.3 Apriori's Sensitivity to Parameters

How well Apriori performed really hinged on setting the right minimum support and confidence thresholds.
- A high min_support threshold produces too few rules, missing meaningful patterns.
- A low min_support threshold generates excessive noise, diluting relevant associations.
- Apriori requires domain knowledge to tune support and confidence thresholds effectively, limiting its scalability without manual intervention.

2.4 Scalability Problems

Both cosine similarity and Apriori face significant scalability challenges as data grows, making them impractical for large-scale e-commerce platforms without substantial optimization or architectural changes.

2.5 Lack of Context Awareness

Right now, the system only looks at what items users interact with. It ignores stuff like:
- When the purchase happened
- Seasonal spikes
- What users are actually looking for at the moment

Recommendations barely shift in real time and miss out on personal touches that keep users engaged.

3. Tradeoffs Identified

The project highlights several important tradeoffs inherent in recommender system design:

Aspect	Tradeoff
Collaborative Filtering	Accuracy vs Cold Start
Association Rule Mining	Interpretability vs Computational Cost
Recommendation System	Accuracy vs Diversity
Model Simplicity	Ease of implementation vs. scalability

There is no single optimal solution — the choice depends on business priorities such as accuracy, scalability, or user engagement.

4. Business Recommendations

4.1 Hybrid Recommendation System - Mix collaborative filtering with association rule mining. Collaborative filtering tailors suggestions to each user, while association rules help with product bundling and encouraging cross-selling. Together, they make recommendations more accurate and reach a wider audience, plus they cut down on cold-start problems.

4.2 Add Content-Based Filtering - Tap into product info like categories and descriptions to recommend items, even when there's not much user data. This helps solve the cold-start issue and makes the system sturdier.

4.3 Boost Diversity and Novelty - Give less popular or newer products a chance by re-ranking recommendations and reducing how often the same items pop up. This approach keeps users engaged and gets them exploring more options.

4.4 Use Scalable Algorithms - Speed things up by switching to quicker algorithms. Go with Approximate Nearest Neighbors instead of calculating full cosine similarity, and pick FP-Growth over Apriori for mining rules. You get better efficiency and real-time results.

4.5 Include Contextual and Behavioral Data - Make recommendations smarter by using what's happening right now: spot trends over time, look at what users do in their sessions, and catch their recent activity. This creates recommendations that actually fit the moment.

4.6 Keep Tracking Business Results - Don't just watch technical stats. Factor in things like click-through rate, conversions, and how recommendations affect revenue. This way, you're sure the system supports real business goals.

Conclusion

In conclusion, collaborative filtering and association rule mining show strong potential for personalization and pattern discovery. User similarity proved valuable for generating relevant recommendations, and association rule mining successfully identified product bundling and cross-selling opportunities. However, both methods are most effective when user data is abundant and evenly distributed, as sparse interaction data consistently weakens output quality across both techniques.

The core challenges encountered include popularity bias, cold start limitations, and scalability constraints that make these methods difficult to deploy at scale without significant optimization. Apriori's performance also depends heavily on manual parameter tuning, adding friction to practical use. Moving forward, a hybrid approach combining collaborative filtering, content-based filtering, and scalable algorithms like FP-Growth and Approximate Nearest Neighbors, alongside contextual and behavioral signals, would produce a more robust and business-ready system.